Software Engineering Assignment (CO311)

B.Tech CSE 5th Sem Autumn 2025

Topic: Problem statement on Hospital Management System

Submitted By:

CSB23010 Kishan Harlalka

CSB23012 Rahul Yadav

CSB23016 Ruknabh Bhattacharyya

CSB23017 Raktim Ranjan Handique

HOSPITAL MANAGEMENT SYSTEM

PATIENT MANAGEMENT IN HOSPITAL

Patient management in a hospital is a complex, systematic process designed to ensure that every patient receives safe, effective, and efficient healthcare from the moment they enter the hospital until after they are discharged. It is a multidisciplinary effort involving clinical, administrative, and support staff.

Problems in Traditional Patient Management:

Traditional processes consist of a series of disconnected, manual steps. A paper file is created at registration; doctors write notes by hand; nurses maintain a separate chart; laboratory requests are sent through physical forms; and billing staff manually compile charges later. This fragmented approach keeps information siloed, slows communication, and increases the risk of errors.

Requirements in Modern Patient Management:

1.Patient Portal: A secure portal/app for patients to book, reschedule, or cancel appointments, pre-register by filling out forms online to reduce waiting time, View lab results, discharge summaries, and prescriptions, make online payments and communicate securely with the care team.

2.Digital Front Desk: Queue Management System to display wait times and manage patient flow, Automated insurance verification, Digital consent forms.

3.Centralized Patient Record**:** A comprehensive digital chart accessible to authorized staff, containing demographics, medical history, allergies, medications, test results, and treatment plans.

4. Computerized Physician Order Entry (CPOE): Allows doctors to enter orders for medications, lab tests, and imaging electronically, reducing handwriting errors.

5. Clinical Documentation**:** Templates for admission notes, progress notes, and consultation reports that are easy for doctors and nurses to use.

6. Digital Task Management**:** A centralized list of tasks for each patient (e.g., "administer IV antibiotic at 4 PM," "check blood sugar")

7. Bed Management Dashboard**:** A real-time, visual overview of all beds in the hospital—showing which are occupied, available, or being cleaned.

8**.**Electronic Medication Administration Record (eMAR): Nurses can scan a patient's wristband and the medication's barcode to ensure the "Five Rights" of medication administration (Right Patient, Right Drug, Right Dose, Right Route, Right Time).

9.Laboratory Information System (LIS) Integration**:** Seamlessly place lab orders and receive results directly into the Central Patient Record being maintained.

10.Radiology Information System (RIS) & PACS Integration**:** Order imaging studies and allow clinicians to view images (X-rays, CTs) from within the patient's chart.

11.Billing Integration**:** Automatically capture charges for procedures, tests, and medications as they are ordered and administered, ensuring accurate and timely billing.

Software Requirements Specification (SRS)

Project: Digital Front Desk (DFD) — Private Hospitals (India)

Document ID: SRS-DFD-001

Version: 1.0

Date: 05/10/2025

Prepared by: Kishan Harlalka CSB23010

Rahul Yadav CSB23012

Ruknabh Bhattacharyya CSB23016

Raktim Ranjan Handique CSB23017

Reviewed by: [Reviewer Name]

Approved by: [Approver Name]

Table of Contents

1. Introduction 1.1 Purpose 1.2 Scope 1.3 Audience 1.4 Definitions, Acronyms, Abbreviations 1.5 References
2. Overall Description 2.1 Product Perspective 2.2 Product Functions (High level) 2.3 User Classes and Characteristics 2.4 Operating Environment 2.5 Design & Implementation Constraints 2.6 Assumptions & Dependencies
3. Specific Requirements 3.1 Functional Requirements (by module) 3.2 Non-Functional Requirements 3.3 External Interface Requirements 3.4 Security & Privacy Requirements 3.5 Performance Requirements
4. System Architecture & Integration 4.1 Integration Standards 4.2 Data Flows 4.3 API Contracts (summary)
5. Workflows & Use Cases 5.1 Primary Workflows 5.2 Use Case Matrix
6. Data Model & Reports 6.1 Key Data Entities 6.2 Sample ERD (textual) 6.3 Standard Reports & Dashboards
7. Validation, Acceptance & Test Criteria 7.1 Acceptance Criteria 7.2 Example Test Cases
8. Compliance, Legal & Localization 8.1 NDHM / ABDM Requirements 8.2 NABH & Clinical Establishments Act 8.3 Data Residency & Retention
9. Deployment & Operational Requirements 9.1 Deployment Topologies 9.2 Backup/Recovery & Monitoring 9.3 Support & Maintenance
10. Appendices 10.1 Revision History

1. Introduction

1.1 Purpose

This SRS describes the functional and non-functional requirements for the **Digital Front Desk (DFD)** system intended for private hospitals in India. The document follows IEEE-style SRS conventions and is intended to be used by product managers, developers, QA, integration engineers, and deployment teams.

1.2 Scope

The DFD is a receptionist-focused module that provides appointment & OPD management, patient registration (OPD/IPD entry), token/queue management, billing initiation, insurance verification, basic EMR access, lab & pharmacy booking, and automated patient communications. The system integrates with an existing Hospital Information System (HIS), third-party LIS/PIS, payment gateways, and NDHM/ABDM services.

1.3 Audience

- Product Owner
- Project Manager
- Developers & Integration Engineers
- QA/Testers
- Deployment/Operations
- Hospital IT & Reception staff

1.4 Definitions, Acronyms, Abbreviations

- DFD: Digital Front Desk
- HIS: Hospital Information System
- MRN: Medical Record Number
- ABHA / Health ID: NDHM patient identifier
- HL7 / FHIR: Interoperability standards
- NABH: National Accreditation Board for Hospitals
- NDHM / ABDM: National Digital Health Mission / Ayushman Bharat Digital Mission
- API: Application Programming Interface

1.5 References

- Company integration guidelines
- Hospital's existing HIS API docs
- NDHM / ABDM interoperability guidelines

- NABH IT requirements

---

2. Overall Description

2.1 Product Perspective

The DFD is a modular, web-based application (responsive) used by receptionists on desktops or tablets. It is not a full HIS; instead it integrates and synchronizes with the hospital's HIS to perform front-desk tasks and to avoid duplicate data entry.

2.2 Product Functions (High level)

- Appointment & OPD management
- Walk-in registration and token issuance
- OPD/IPD registration and MRN lifecycle
- Billing initiation (preliminary) and payment capture
- Insurance & scheme verification and claim initiation
- Basic EMR access (view-only at front desk) and link to HIS
- Lab & pharmacy order placement (triggering LIS/PIS workflows)
- Queue display & token announcement
- SMS/WhatsApp/email notifications
- Reports and dashboards for reception managers

2.3 User Classes and Characteristics

Role	Primary Tasks	Technical Proficiency
Reception ist	Register patients, issue tokens, initiate billing, manage queues	Basic computer literacy
Front-desk Superviso r	Configure schedules, view reports, manage staff	Intermediate
HIS Admin / Integratio n Engineer	Configure connectors, monitor synchronization	Advanced
Admin / Finance	Reconcile payments, export MIS	Advanced

2.4 Operating Environment

- Server: Linux (Ubuntu 24.04 LTS recommended) or cloud provider region within India
- Backend: Node.js / Java / .NET (TBD by project)
- Database: PostgreSQL / MySQL
- Client: Chrome/Edge browsers on desktop/tablet, touch screen support
- Connectivity: HTTPS-based connections to HIS, NDHM, payment gateways

2.5 Design & Implementation Constraints

- Must integrate with the hospital's existing HIS; cannot assume HIS replacement
- Must use supported standards (HL7/FHIR) where HIS supports them
- Must maintain logs/audit trails for regulatory audits
- Data retention & residency within India

2.6 Assumptions & Dependencies

- HIS provides an API or middleware for integration
- OCR/scanning devices for ABHA QR or ID cards are available
- Payment gateway and SMS/WhatsApp providers are provisioned by hospital

---

3. Specific Requirements

3.1 Functional Requirements (by module)

Each functional requirement is uniquely identified (FR-###), includes priority, description, and acceptance criteria.

3.1.1 Appointment & OPD Management

- **FR-001 — Appointment Creation (High):** System shall allow receptionist to create, reschedule, cancel appointments and mark walk-ins. *Acceptance:* Appointment reflected in DFD and synced to HIS within 5 seconds.
- **FR-002 — Appointment View (High):** System shall display day's schedule with filters by doctor, specialty, and time slots.
- **FR-003 — No-show Handling (Medium):** Mark no-shows and optionally trigger follow-up reminders.

3.1.2 Patient Registration (OPD/IPD)

- **FR-010 — New Patient Registration (High):** Capture demographics, contact, ID proofs, assign MRN, and optionally link/generate ABHA. *Acceptance:* MRN created and visible in HIS; ABHA validation returns success or clear error.

- **FR-011 — Update Patient Details (Medium):** Edit non-critical fields with audit trail.
- **FR-012 — IPD Admission Initiation (Medium):** For admissions, capture bed/ward, admitting consultant, provisional diagnosis; push admission ADT message to HIS.

3.1.3 Queue & Token Management

- **FR-020 — Token Generation (High):** Auto-generate sequential tokens for appointments and walk-ins; support QR and printed token. *Acceptance:* Token number issued and queue updated in <2s.
- **FR-021 — Token Display & Announce (High):** Display current tokens on waiting-room screens; allow manual override and emergency priority.
- **FR-022 — SMS/WhatsApp Token Notification (Medium):** Send token number and estimated wait time when issued.

3.1.4 Billing Initiation

- **FR-030 — Provisional Bill Creation (High):** Create a provisional bill capturing consultation charges and optional lab/pharmacy pre-charges.
- **FR-031 — Multi-Mode Payment Capture (High):** Accept payments via cash, card, UPI, and record partial payments/deposits.
- **FR-032 — Integration with Billing Engine (High):** Push provisional bill to HIS/Billing module and reconcile real-time payment status.

3.1.5 Insurance & Scheme Verification

- **FR-040 — Insurance Eligibility Check (High):** Verify insurance / TPA eligibility via API. *Acceptance:* Eligibility response in <5s.
- **FR-041 — AB-PMJAY / Government Schemes (High):** Initiate required pre-authorization/claim initiation steps per scheme.

3.1.6 Basic EMR Access

- **FR-050 — EMR Lookup (Medium):** Search and view recent visits, outstanding reports, and allergies (read-only at front desk).

3.1.7 Lab & Pharmacy Integration

- **FR-060 — Lab/Pharmacy Order Placement (Medium):** Create lab test booking or pharmacy order and push to LIS/PIS.
- **FR-061 — Receive Test Status (Low):** Show test status (sample collected, result ready) from LIS.

3.1.8 Reporting & Dashboards

- **FR-070 — Daily Summary Report (High):** Generate counts for registrations, bill totals, no-shows, average wait time.
- **FR-071 — Export MIS (Medium):** Export reports to CSV/PDF.

3.1.9 Patient Communications

- **FR-080 — Notifications (High):** Send appointment reminders and token calls via SMS/WhatsApp/email.
- **FR-081 — Templates (Medium):** Admin can configure message templates and opt-in preferences.

3.2 Non-Functional Requirements

3.2.1 Usability

- **NFR-001:** Receptionist dashboard shall load within 1.5s on local LAN.
- **NFR-002:** UI shall support English and configurable regional languages.

3.2.2 Reliability & Availability

- **NFR-010:** System uptime >= 99.5% (excluding scheduled maintenance).

3.2.3 Security

- **NFR-020:** All traffic encrypted using TLS 1.2 or higher.
- **NFR-021:** Passwords hashed with strong algorithm (bcrypt/argon2).
- **NFR-022:** Role-based access control; audit logs for create/update/delete operations.

3.2.4 Performance

- **NFR-030:** Token issuance and queue update should reflect across all clients in under 2 seconds on LAN.
- **NFR-031:** System shall handle 100 concurrent receptionist sessions per hospital instance.

3.2.5 Maintainability & Extensibility

- **NFR-040:** Provide well-documented REST APIs and connector interfaces for HIS, LIS, and third-party services.

3.2.6 Localization & Formats

- **NFR-050:** Date/time formats default to DD/MM/YYYY and currency in INR. Support input of Aadhar/PAN IDs where needed.

3.3 External Interface Requirements

3.3.1 User Interfaces

- Responsive web UI (desktop/tablet)
- Waiting-room display screen (read-only) for token call
- Optional kiosk UI for self-check-in

3.3.2 Hardware Interfaces

- ABHA QR scanner
- Barcode/ID scanners
- Card reader (optional) for ID verification
- Printer for tokens/receipts

3.3.3 Software Interfaces

- HIS: HL7/FHIR/REST APIs for patient, appointment, billing
- NDHM / ABHA: OAuth2-secured REST endpoints for ABHA validation and PHR updates
- LIS/PIS: HL7/FHIR integration for orders/results
- Payment Gateway: Standard gateway APIs (PCI-DSS compliance recommended)
- SMS / WhatsApp gateway: REST APIs

3.4 Security & Privacy Requirements

- Patient consent must be recorded for any PII or clinical data exchange (per NDHM consent framework).
- Access control: least privilege for receptionist; elevated roles for finance/HIS admin.
- Audit trail for all data access/modification with timestamp, user-id, and IP.
- Data at rest encryption for sensitive fields (PHI) as per hospital policy.

3.5 Performance Requirements

- End-to-end ABHA validation should complete within 5 seconds under standard load.
- Background sync jobs (e.g., reconcile payments) may run asynchronously and must not block receptionist operations.

---

4. System Architecture & Integration

4.1 Integration Standards

- HL7 v2.x for legacy HIS adapters (ADT, ORM, ORU)
- FHIR R4 for NDHM and modern HIS interactions (Patient, Encounter, Observation, MedicationRequest)
- OAuth2 and TLS for authentication/authorization to NDHM and third-party services
- JSON over HTTPS for REST APIs

4.2 Data Flows

- Registration -> MRN generation -> ABHA validation -> Push to HIS (ADT)
- Appointment booking -> Token generation -> Queue display -> Doctor call -> Billing initiation -> Finalize bill in HIS
- Lab order -> LIS -> Sample status -> Result posted -> EMR updated

4.3 API Contracts (summary)

Provide OpenAPI/Swagger specs for these endpoints (examples): - POST /api/patients (create/update patient) - GET /api/patients/{mrn} - POST /api/appointments - POST /api/tokens - POST /api/billing/provisional - POST /api/integrations/ndhm/validate-abha

---

5. Workflows & Use Cases

5.1 Primary Workflows

1. **Walk-in Patient Registration & Token:** Receptionist scans ID/ABHA or enters data -> system creates MRN -> token issued -> queue updated -> SMS confirmation sent.
2. **Appointment Check-in:** Patient arrives, receptionist confirms appointment -> check-in -> token assigned -> patient waits.
3. **Billing Initiation:** After consultation, receptionist creates provisional bill -> captures payment partial/full -> pushes final bill to HIS accounting.

5.2 Use Case Matrix (high-level)

Use Case	Actor	Trigger	Precondition	Postcondition
Register patient	Receptionist	Walk-in / New appointment	HIS reachable	MRN created and stored
Issue token	Receptionist / Kiosk	Registration or check-in	MRN exists	Token issued; queue updated
Verify insurance	Receptionist	During billing	Provider TPA API available	Eligibility result obtained

---

6. Data Model & Reports

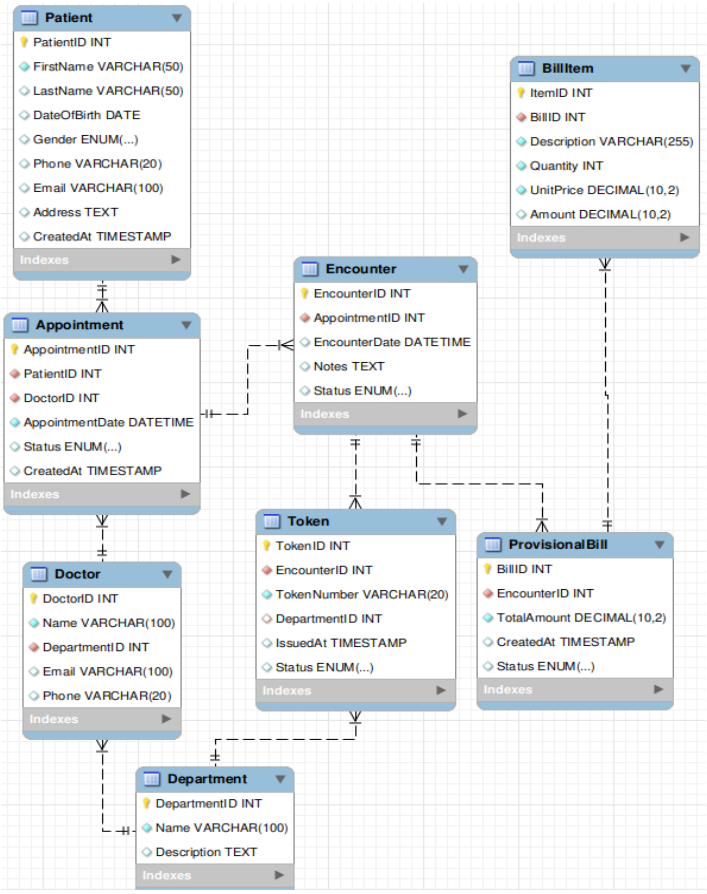
6.1 Key Data Entities

- Patient (MRN, name, contact, ABHA, demographics)

- Appointment (id, patient, doctor, timeslot, status)
- Token (token_no, type, status, issued_at)
- Encounter (visit id, department, consultant)
- ProvisionalBill (id, items, total, payments)
- LabOrder / PharmacyOrder

6.2 Sample ERD (textual & diagram)

- Patient 1 — * Appointment
- Appointment 1 — 1 Encounter
- Encounter 1 — * Token
- Encounter 1 — 1 ProvisionalBill



6.3 Standard Reports & Dashboards

- Daily front-desk summary (registrations, billed amount, no-shows)
- Average wait time per department
- Token throughput per hour
- Outstanding payments & deposits

7. Validation, Acceptance & Test Criteria

7.1 Acceptance Criteria (examples)

- **AC-01:** New patient registration successfully creates MRN in DFD and syncs to HIS within 5s.
- **AC-02:** Token issuance displays on all waiting-room screens within 2s.
- **AC-03:** ABHA validation returns status (valid/invalid) with error details when invalid.
- **AC-04:** Provisional bill can be created and a partial payment recorded; status reconciles with HIS billing.

7.2 Example Test Cases

- **TC-001:** Register patient with ABHA QR — expect MRN + ABHA link.
- **TC-002:** Create appointment for Dr. X at 10:30 — expect appointment visible in day view.
- **TC-003:** Issue token and verify waiting-room display updates.
- **TC-004:** Simulate insurance API failure — ensure graceful fallback and error message.

---

8. Compliance, Legal & Localization

8.1 NDHM / ABDM Requirements

- ABHA generation / linking flow integrated at registration
- Consent capture and storage per NDHM consent artifact model
- Use NDHM FHIR profiles for PHR updates where applicable

8.2 NABH & Clinical Establishments Act

- Maintain audit trails, secure storage, and logs for required durations
- Support standard operating workflows required by NABH accreditation (consent, patient identification, incident logging)

8.3 Data Residency & Retention

- Production data stored in India (cloud region or on-premise)
- Retention policy: patient records retained per hospital policy/state law (e.g., 7 years typical; configurable)

---

9. Deployment & Operational Requirements

9.1 Deployment Topologies

- **Single-site deployment:** On-premise servers with local LAN for front-desk clients; cloud connectors for NDHM
- **Cloud-hosted deployment:** All services hosted in India cloud region (private VPC) with secure VPN to HIS if HIS is on-prem.

9.2 Backup/Recovery & Monitoring

- Daily database backups + point-in-time recovery for last 7 days
- Monitoring (uptime, queue lengths, failed integrations) with alerting to hospital IT

9.3 Support & Maintenance

- 24x6 support SLA (configurable)
- Regular updates for NDHM/HL7/FHIR changes

---

10. Appendices

10.1 Revision History

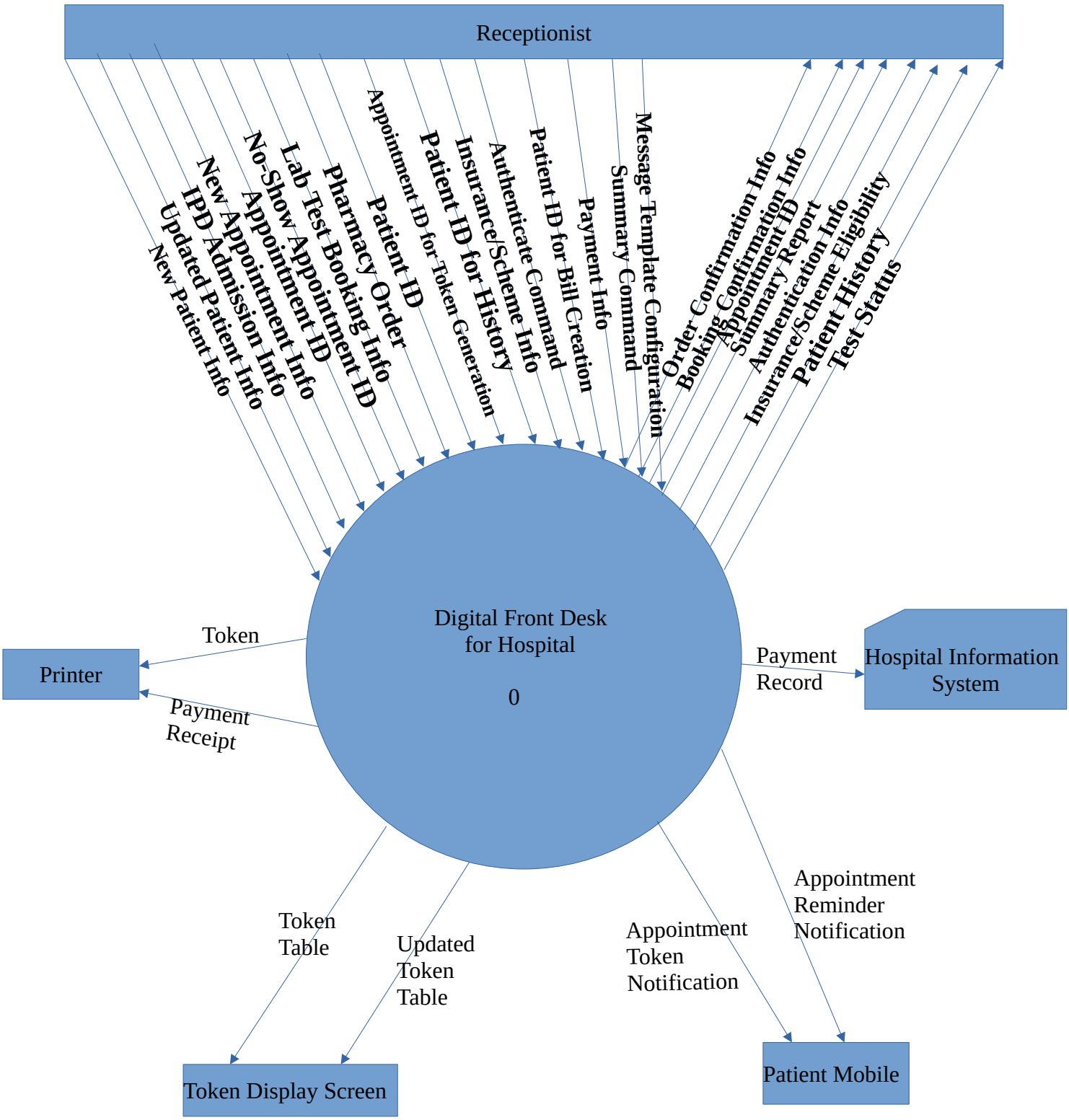
Version	Date	Author	Change
1.0	05/10/2025	Kishan Harlalka Rahul Yadav Ruknabh Bhattacharyya Raktim Ranjan Handique	Initial draft

---

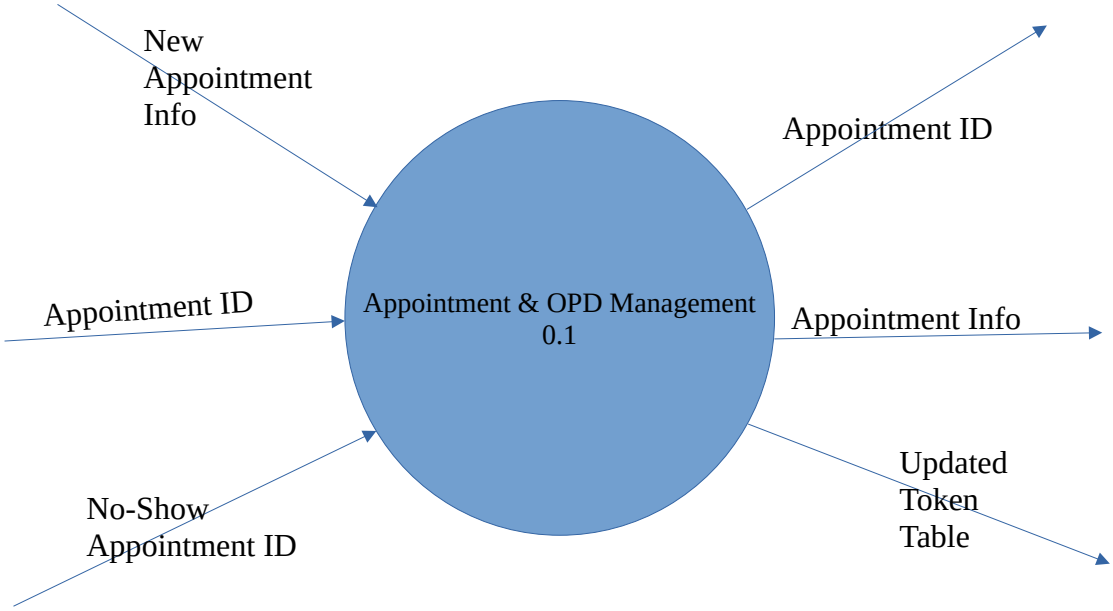
End of SRS Document

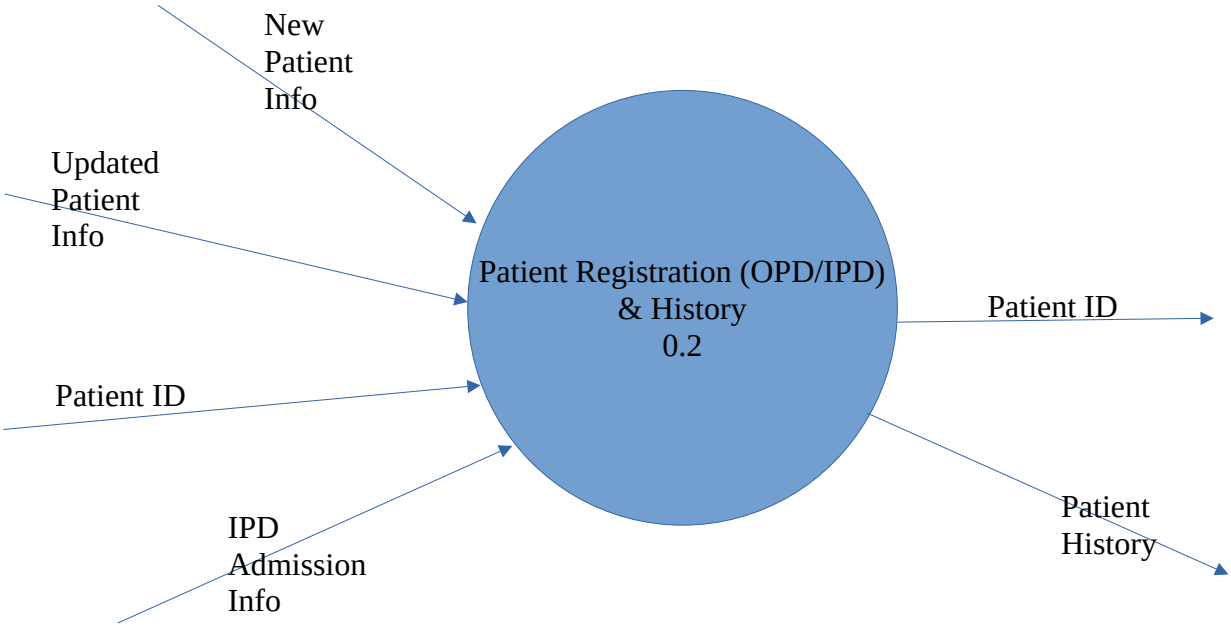
Design Document
(Hospital Digital Front Desk Management)

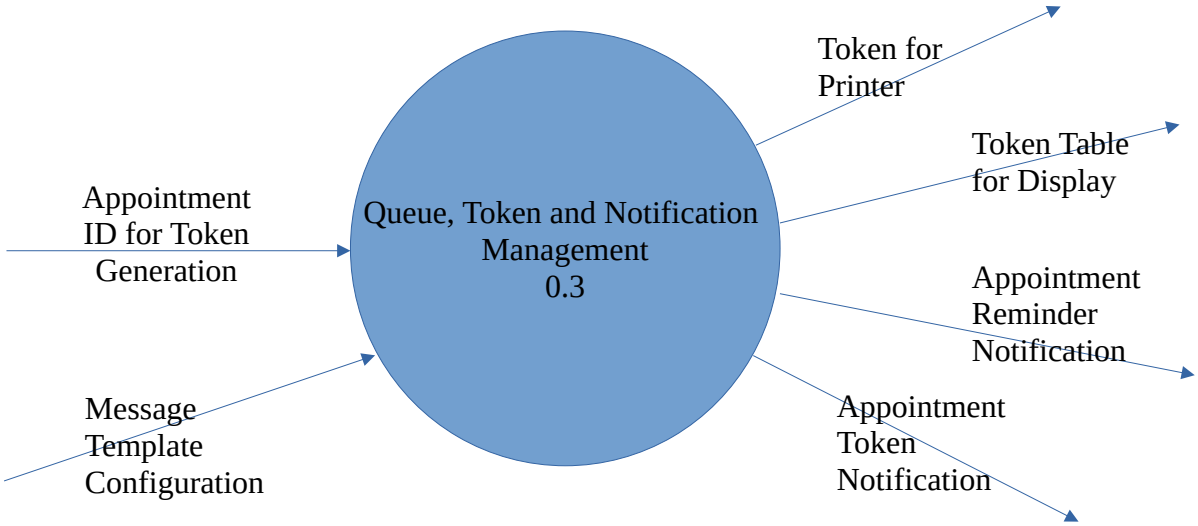
Level 0 Data Flow Diagram :

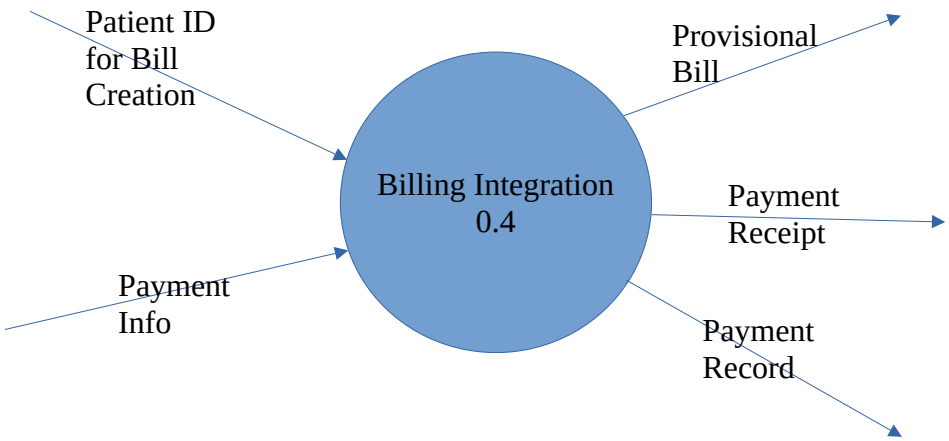


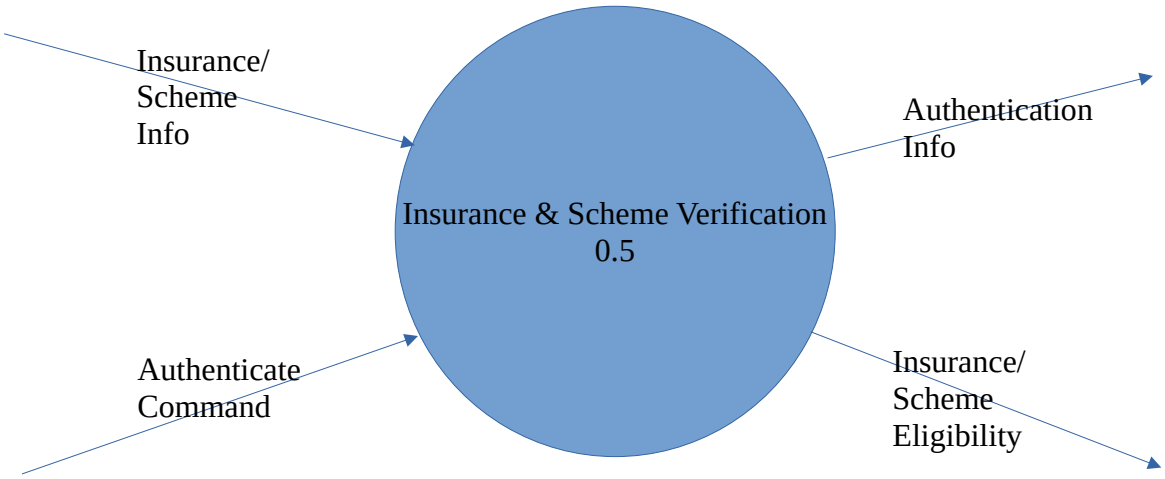
Level 1 Data Flow Diagrams :

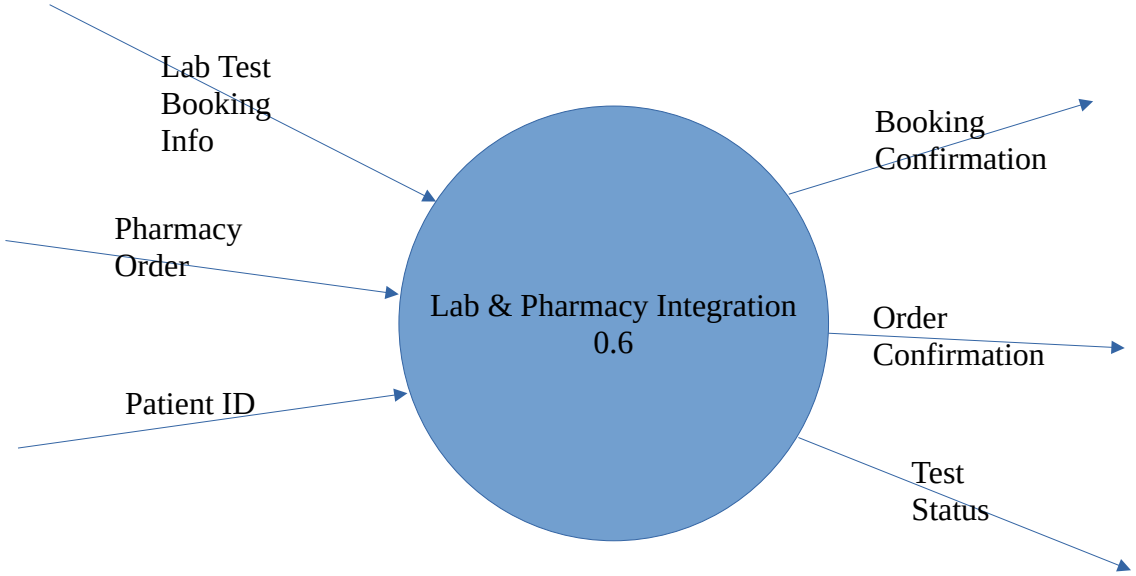


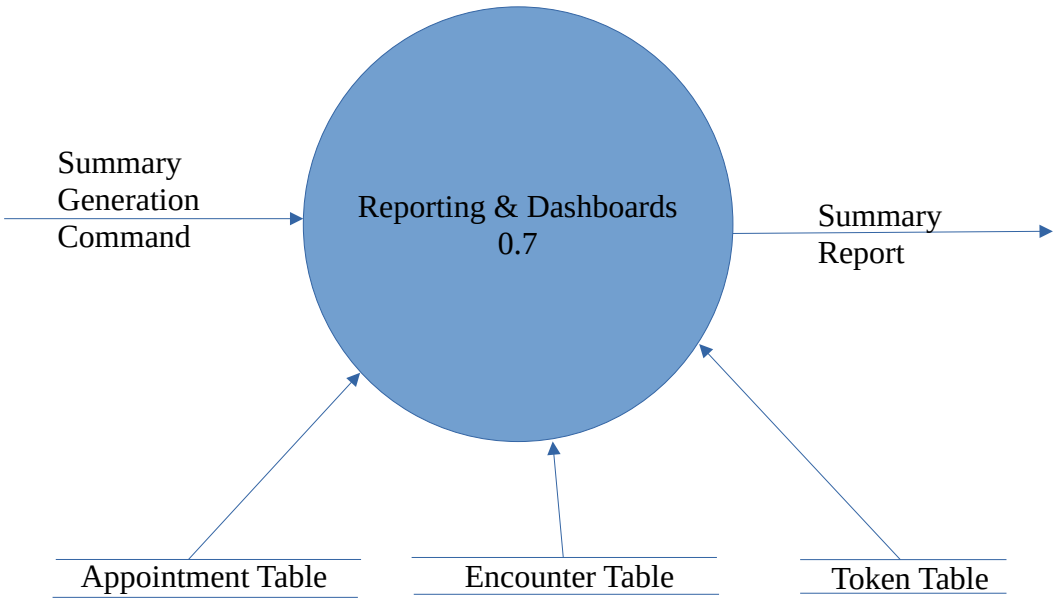




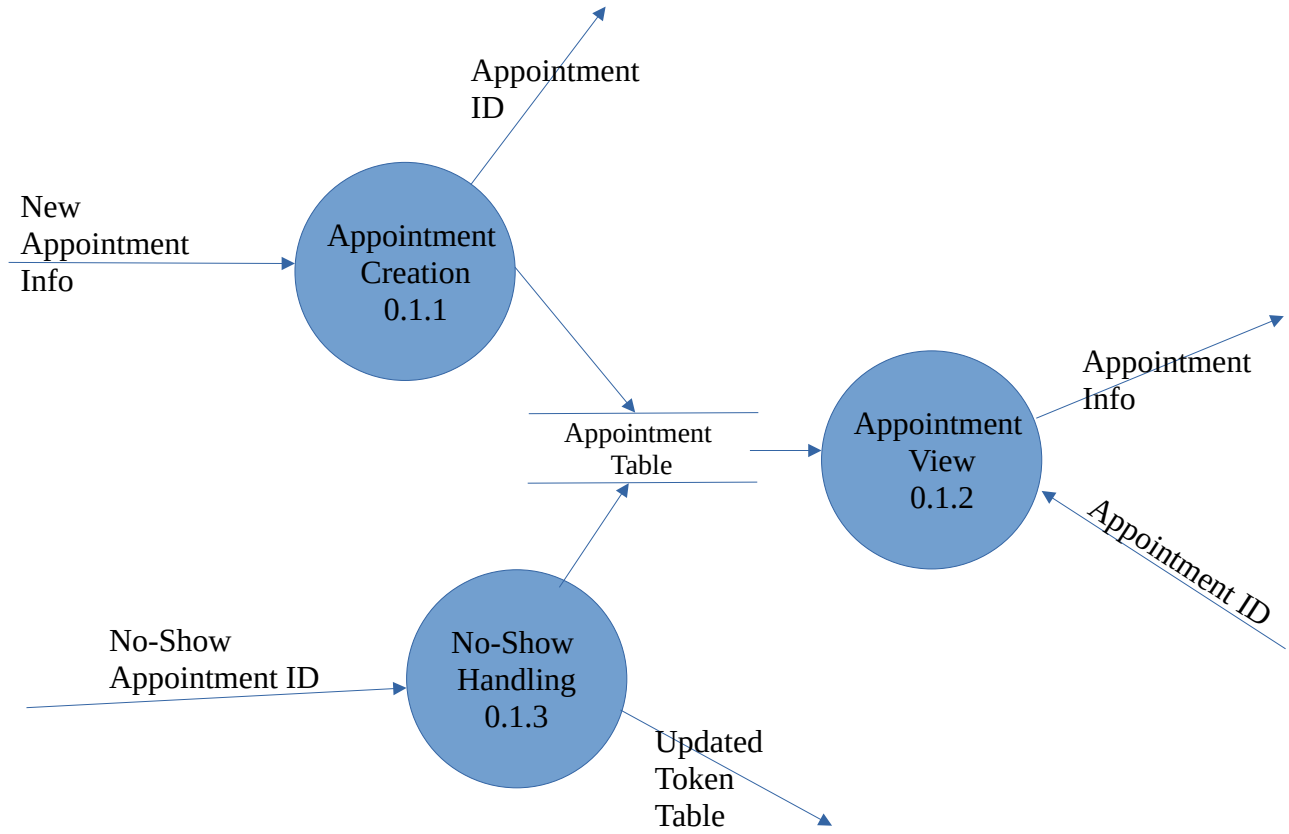

Patient ID
for Bill
Creation
Provisional
Bill
Billing Integration
0.4
Payment
Receipt
Payment
Info
Payment
Record

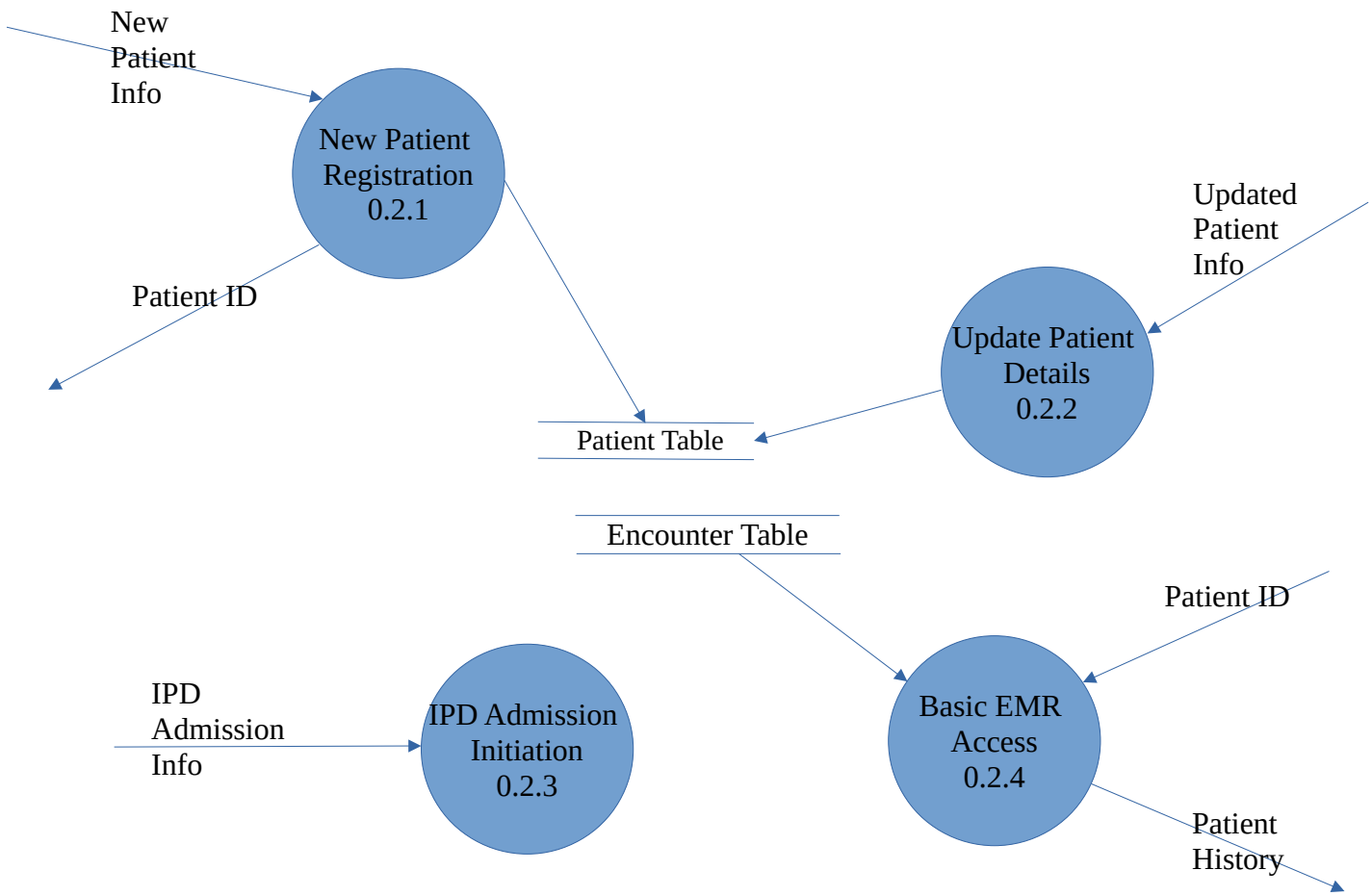

Insurance/
Scheme
Info
Authentication
Info
Insurance & Scheme Verification
0.5
Authenticate
Command
Insurance/
Scheme
Eligibility

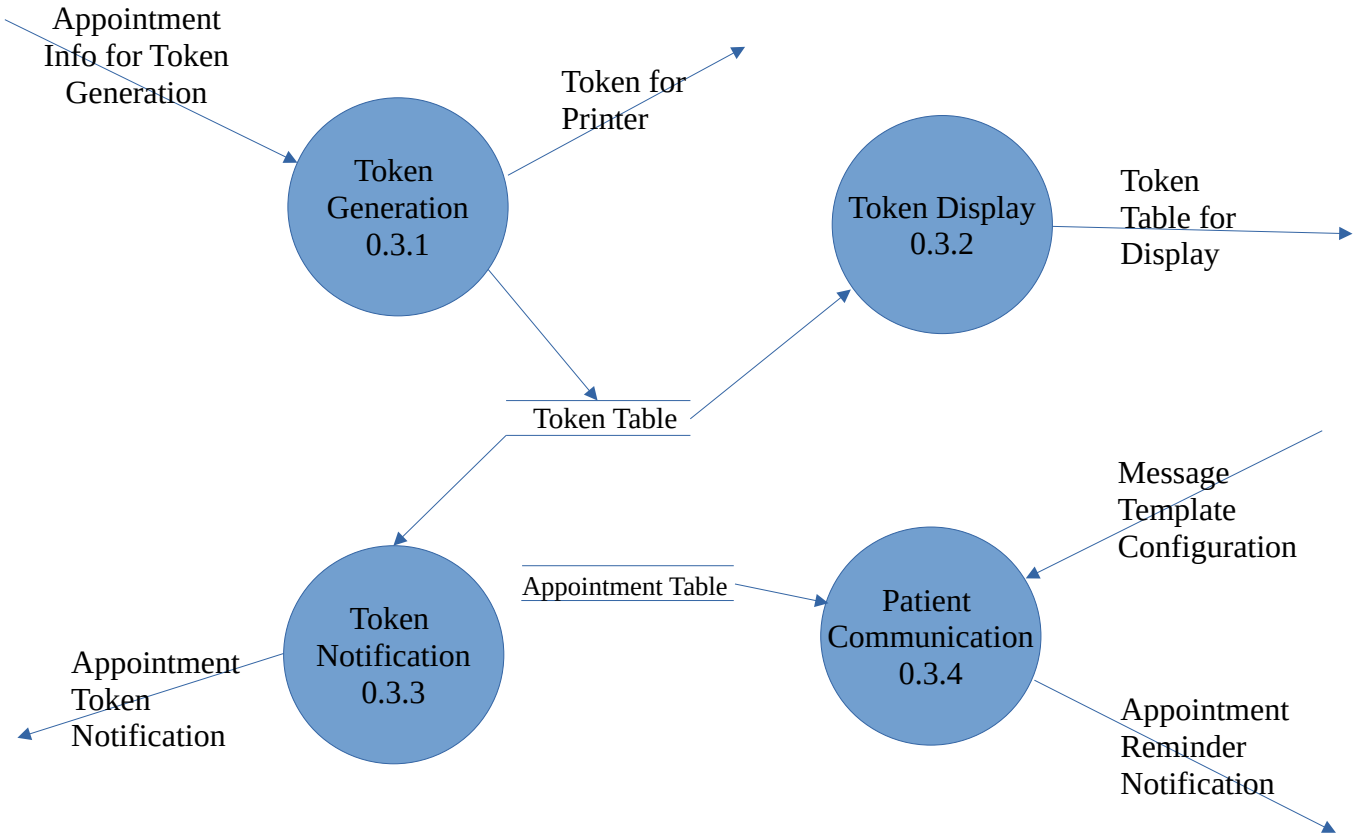

Lab Test Booking Info
Pharmacy Order
Patient ID
Lab & Pharmacy Integration 0.6
Booking Confirmation
Order Confirmation
Test Status

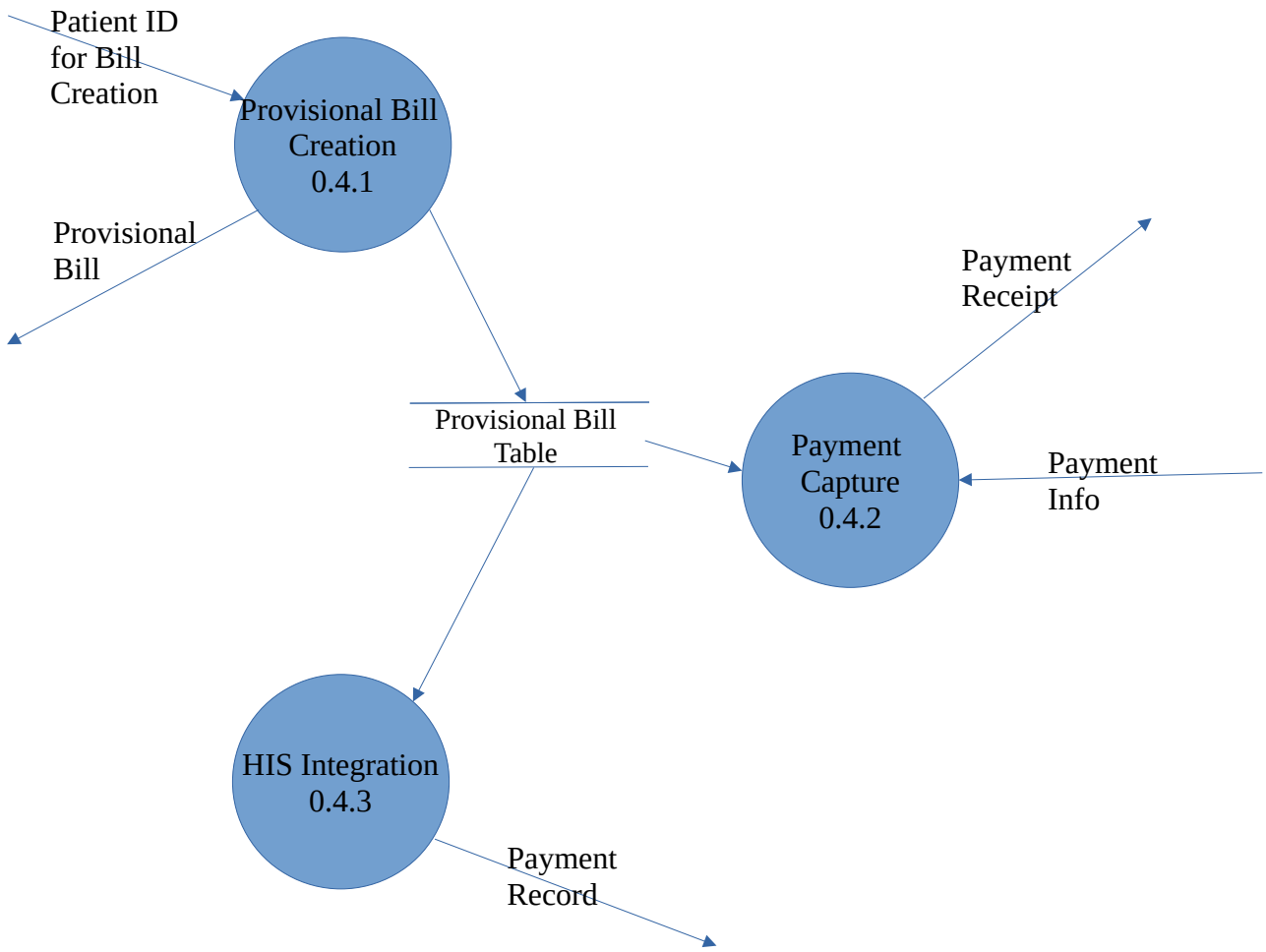

Summary Generation Command
Reporting & Dashboards 0.7
Summary Report
Appointment Table
Encounter Table
Token Table

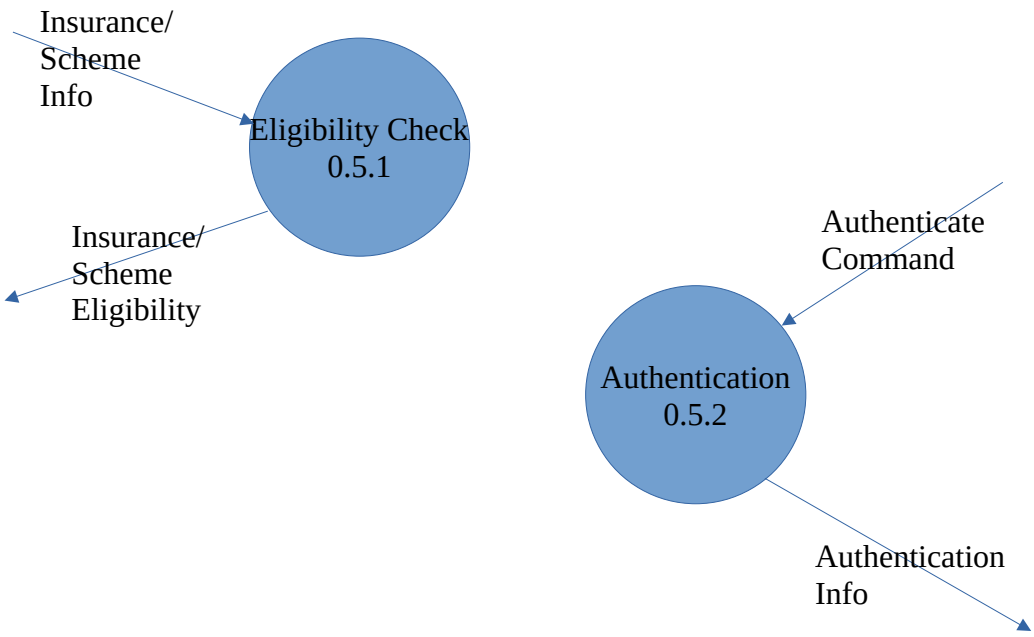
Level 2 Data Flow Diagrams :

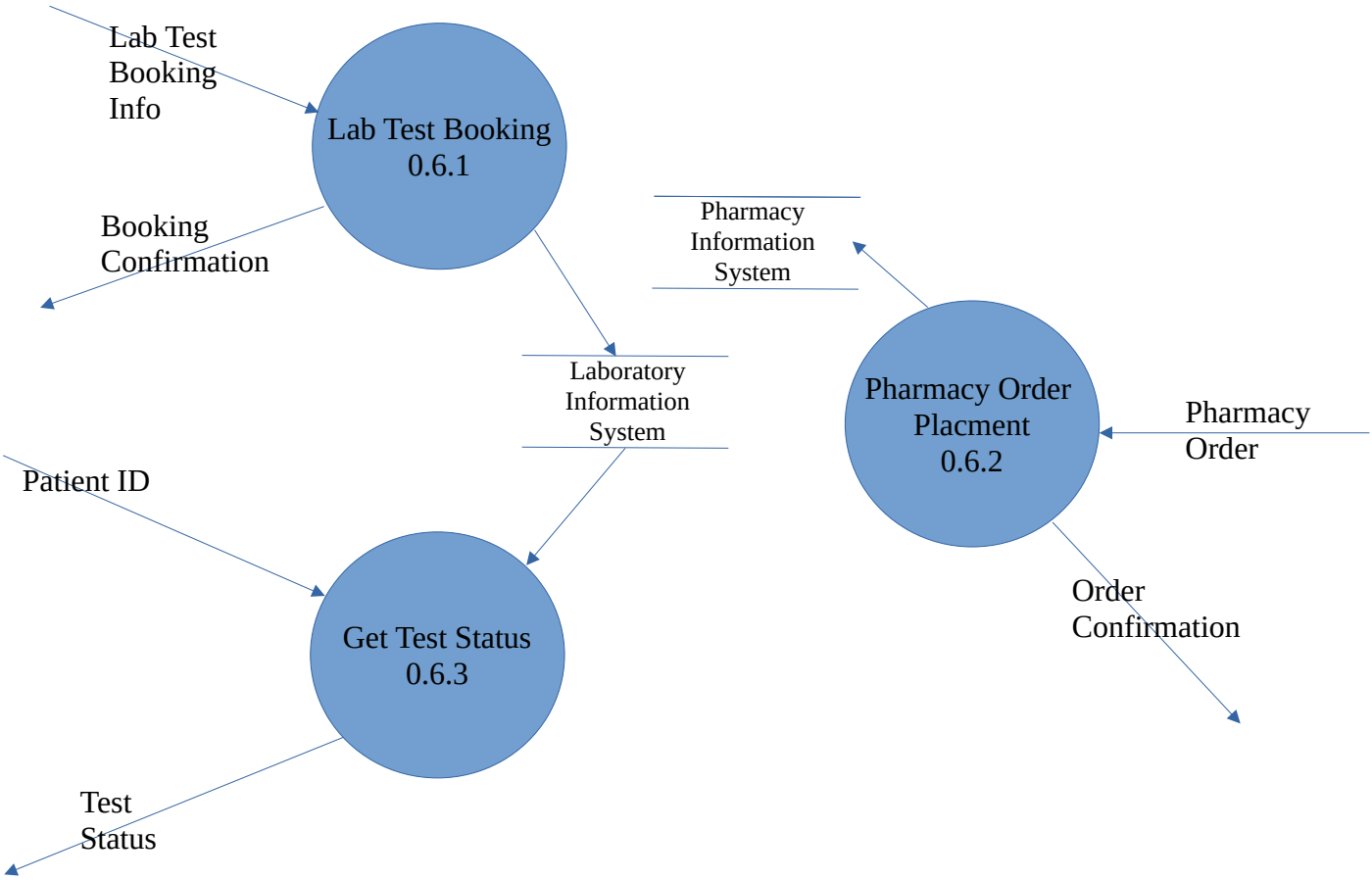




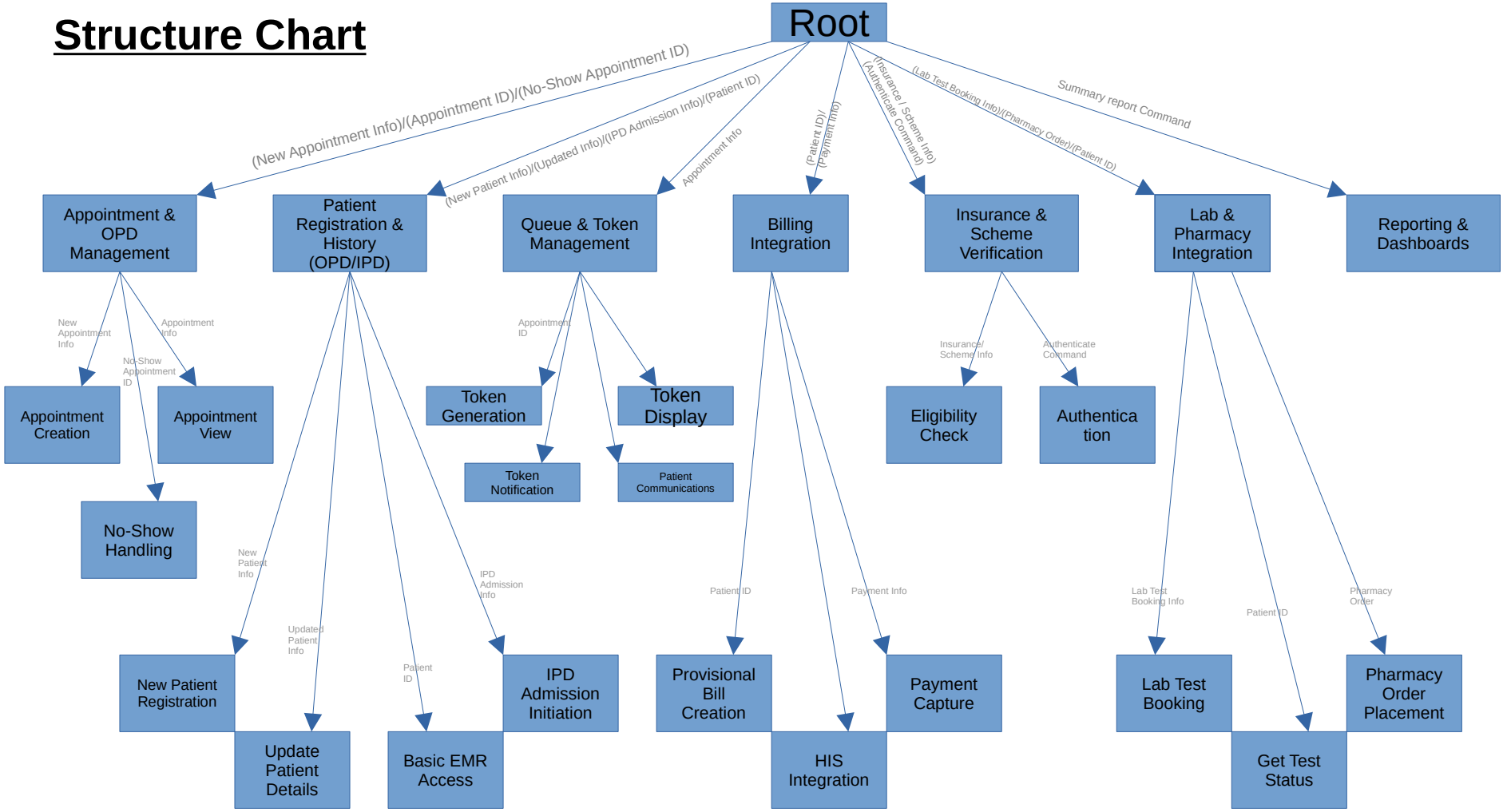

Appointment Info for Token Generation
Token Generation 0.3.1
Token for Printer
Token Display 0.3.2
Token Table for Display
Token Table
Message Template Configuration
Appointment Table
Patient Communication 0.3.4
Token Notification 0.3.3
Appointment Token Notification
Appointment Reminder Notification


Patient ID for Bill Creation
Provisional Bill Creation 0.4.1
Provisional Bill
Payment Receipt
Provisional Bill Table
Payment Capture 0.4.2
Payment Info
HIS Integration 0.4.3
Payment Record


Insurance/
Scheme
Info
Eligibility Check
0.5.1
Insurance/
Scheme
Eligibility
Authenticate
Command
Authentication
0.5.2
Authentication
Info


Lab Test
Booking
Info
Lab Test Booking
0.6.1
Booking
Confirmation
Pharmacy
Information
System
Laboratory
Information
System
Pharmacy Order
Placment
0.6.2
Pharmacy
Order
Patient ID
Get Test Status
0.6.3
Order
Confirmation
Test
Status

Structure Chart



Data Dictionary

Data Decompositions :

```
Appointment_Info = {
    Appointment_ID,
    Patient_ID,
    Doctor_ID,
    Appointment_Date,
    Created_At
}
```

```
New_Appointment_Info = {
    Appointment_ID,
    Patient_ID,
    Doctor_ID,
    Appointment_Date
}
```

```
Token_Table = {
    Appointment_ID,
    Appointment_Time
}
```

```
Updated_Token_Table = {
    Appointment_ID,
    Appointment_Time
}
```

```
New_Patient_Info = {
    Patient_ID,
    Patient_First Name,
    Patient_Last_Name,
    Patient_Gender,
    Patient_Date_Of_Birth,
    Patient_Phone,
    Patient_Email,
    Patient_Address
}
```

```
Updated_Patient_Info = {
    Patient_ID,
    Patient_First Name,
    Patient_Last_Name,
    Patient_Gender,
    Patient_Date_Of_Birth,
    Patient_Phone,
    Patient_Email,
    Patient_Address
}
```

```
IPD_Admission_Info = {
    Admission_ID,
    Patient_ID,
    Admission_Date,
    Department,
    Doctor_Assigned,
    Admission_Reason,
    Initial_Diagnosis,
    Payment_Mode,
    Advance_Amount
}
```

```
Payment_Info = {
    Encounter_ID,
    Payment_Type,
    Transaction_Number,
    Amount,
}
```

```
Provisional_Bill = {
    Bill_ID,
    Encounter_ID,
    Amount,
    Created_At
}
```

```
Payment_Receipt = {
    Receipt_ID,
    Payment_ID,
    Bill_ID,
    Patient_ID,
    Payment_Date,
    Payment_Mode,
    Amount,
    Received_By,
    Remarks
}
```

```
Payment_Record = {
    Receipt_ID,
    Payment_ID,
    Bill_ID,
    Patient_ID,
    Payment_Date,
    Payment_Mode,
    Amount,
    Received_By,
    Remarks
}
```

```
Insurance_or_Scheme_Info = {
    Insurance_ID,
    Patient_ID,
    Coverage_Type,
    Provider_Name,
    Policy_Number,
    Scheme_Name,
    Valid_From,
    Valid_Till,
    Total_Coverage_Amount,
    Verification_Date,
}
```

```
Authentication_Info = {
    Auth_ID,
    Insurance_ID,
    Patient_ID,
    Request_Timestamp,
    Auth_Status,
    Verification_Timestamp
}
```

```
Patient_History = {
    History_ID,
    Patient_ID,
    Chronic_Diseases,
    Past_Surgeries,
    Allergies,
    Current_Medications,
    Last_Admission_ID,
    Last_Visit_Date
}
```

```
Lab_Test_Booking_Info = {
    Booking_ID,
    Patient_ID,
    Test_ID_List,
    Booking_Date,
    Scheduled_Date,
    Booking_Mode,
    Payment_Status
}
```

```
Pharmacy_Order = {
    Order_ID,
    Patient_ID,
    Ordered_Medicines,
    Order_Date,
    Order_Mode,
    Payment_Status,
    Dispensed_By,
    Dispense_Status,
    Total_Amount
}
```

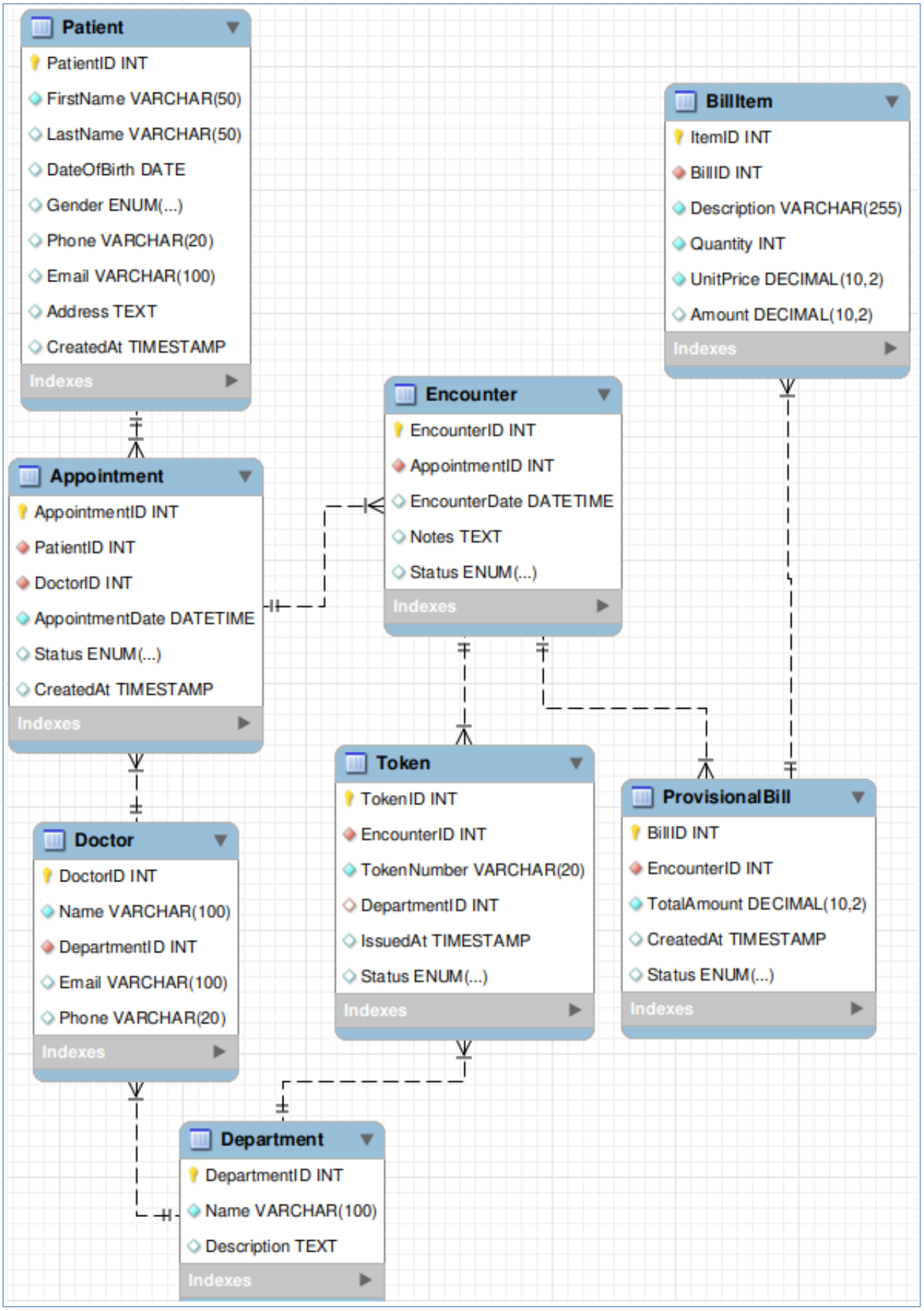
```
Test_Status = {
    Status_ID,
    Booking_ID,
    Test_ID,
    Sample_ID,
    Status,
    Status_Timestamp,
}
```

```
Summary_Report = {
    Report_ID,
    Report_Date,
    Department_ID,
    Doctor_ID,
    Total_Appointments,
    Completed_Encounters,
    Pending_Encounters,
    Cancelled_Appointments,
    Average_Wait_Time,
    Peak_Hours,
    First_Token_No,
    Last_Token_No,
}
```

Data Descriptions:

Data Name	Description	Type
Appointment_ID	Unique identifier for appointment	Integer
Patient_ID	Unique identifier for patient	Integer
Doctor_ID	Unique identifier for doctor	Integer
Appointment_Date	Date of appointment	DateTime
Created_At	Timestamp when appointment created	DateTime
No_Show_Appointment_ID	Identifier for missed appointment	Integer
Appointment_Time	Scheduled appointment time	Time
Patient_First_Name	Patient first name	String
Patient_Last_Name	Patient last name	String
Patient_Gender	Gender of patient	String
Patient_Date_Of_Birth	Patient date of birth	Date
Patient_Phone	Patient phone number	String
Patient_Email	Patient email address	String
Patient_Address	Patient residential address	String
Admission_ID	Unique identifier for hospital admission	Integer
Admission_Date	Date of hospital admission	Date
Department	Department handling the admission	String
Doctor_Assigned	Doctor assigned for the case	String
Admission_Reason	Reason for hospital admission	String
Initial_Diagnosis	Preliminary diagnosis at admission	String
Payment_Mode	Mode of payment (Cash/Card/Insurance)	String
Advance_Amount	Initial advance amount paid	Decimal
Encounter_ID	Unique identifier for patient encounter	Integer
Payment_Type	Type of payment (Cash/Card/Online)	String
Transaction_Number	Transaction reference number	String
Amount	Amount involved in transaction	Decimal
Bill_ID	Unique identifier for provisional bill	Integer
Created_At	Timestamp when bill created	DateTime
Receipt_ID	Unique identifier for receipt	Integer
Payment_ID	Identifier for payment transaction	Integer
Payment_Date	Date when payment made	Date
Received_By	Staff who received payment	String
Remarks	Additional comments or notes	String

Insurance_ID	Unique identifier for insurance/scheme record	Integer
Coverage_Type	Type of coverage (Private/Govt/Corporate)	String
Provider_Name	Name of insurance provider or scheme	String
Policy_Number	Insurance policy or card number	String
Scheme_Name	Name of insurance or health scheme	String
Valid_From	Start date of insurance validity	Date
Valid_Till	End date of insurance validity	Date
Total_Coverage_Amount	Maximum amount covered under policy	Decimal
Verification_Date	Date of policy verification	DateTime
Auth_ID	Unique identifier for authentication record	Integer
Request_Timestamp	Time when authentication requested	DateTime
Auth_Status	Result of authentication process	String
Verification_Timestamp	Time when authentication completed	DateTime
History_ID	Unique identifier for patient history record	Integer
Chronic_Diseases	List of chronic conditions	List<String>
Past_Surgeries	List of past surgical procedures	List<String>
Allergies	List of known allergies	List<String>
Current_Medications	List of current medications	List<String>
Last_Admission_ID	Reference to last hospital admission	Integer
Last_Visit_Date	Date of last hospital visit	Date
Booking_ID	Unique identifier for lab test booking	Integer
Test_ID_List	List of test identifiers booked	List<Integer>
Booking_Date	Date of booking	DateTime
Scheduled_Date	Scheduled date for sample collection	DateTime
Booking_Mode	Mode of booking (FrontDesk/Online/DoctorRequest)	String
Payment_Status	Status of payment for test	String
Order_ID	Unique identifier for pharmacy order	Integer
Ordered_Medicines	List of ordered medicines with quantity	List<Integer>
Order_Date	Date when order was placed	DateTime
Order_Mode	How order was placed	String
Dispensed_By	Staff who dispensed medicines	String
Dispense_Status	Current dispensing status	String
Total_Amount	Total cost of order	Decimal

Status_ID	Unique identifier for test status record	Integer
Test_ID	Reference to specific test	Integer
Sample_ID	Identifier for collected sample	Integer
Status	Current status of test	String
Status_Timestamp	Time when status last updated	DateTime
Report_ID	Unique identifier for report	Integer
Report_Date	Date of report generation	Date
Department_ID	Department for which report generated	Integer
Doctor_ID	Doctor for which report generated	Integer
Total_Appointments	Total appointments on report date	Integer
Completed_Encounters	Number of encounters completed	Integer
Pending_Encounters	Number of pending encounters	Integer
Cancelled_Appointments	Number of cancelled appointments	Integer
Average_Wait_Time	Average wait time for patients	Time
Peak_Hours	Time range with highest load	String
First_Token_No	First token number of the day	Integer
Last_Token_No	Last token number of the day	Integer

Entity Relationship Diagram